📦 Master Collection Setup Guide — All Projects + Kit (₹2,999)

⚠️ Seller checklist before you sell a single copy: (1) URGENT — move funds out of the exposed Solana wallet and delete the hardcoded private key from all 7 files (Section 0), (2) revoke the leaked ASES GitHub token and strip/rotate every other secret, (3) keep the honesty limits in each section — they are your legal shield, (4) do a clean-room test run of each project yourself. Do NOT skip these.

One-Time Price: ₹2,999 · Full white-label rights (modify, rebrand, use, and resell)

attachment:a5d7a348-4328-4f10-bf45-6a33a3b211de:Master_Collection_Setup_Guide.pdf

Master Collection Setup Guide — full printable PDF

This is the complete, beginner-friendly setup guide for the **Kunal Das Master Collection**. It bundles **GODSHAND Cinematic AI Film Kit**, **Convergence Framework**, **AI Video Monetizer**, **ASES (Autonomous Software Engineering System)**, **SafeScreen Content Moderation**, and **Solana Sniper (paper-trading study + source)**, plus supporting scripts.

How to use this guide

1. Unzip the Master ZIP (provided to buyers).
2. Follow each section; every project is self-contained — jump to what you need.
3. If you are reselling, do Section 0 (strip secrets) before you redistribute anything.

🧭 Honest limits (global): This is not one-click magic. Every project needs setup, API keys, hardware (a GPU for video), and a short learning curve. AI outputs can contain artifacts. Check model/data licenses before commercial use.

---

0. Seller step: strip & rotate secrets (do this FIRST)

🚨 MOST SEVERE — live wallet private key: the Solana Sniper project has a hardcoded Solana wallet private key (base58) in SEVEN plain files that are NOT protected by `.gitignore`: `check_balance.cjs`, `monitor_progress.cjs`, `start_bot.ps1`, `start_bot_daemon.ps1` (twice), `temp.txt`, and `README.md`. Those files also expose a live Helius API key, a live Birdeye API key, and a QuikNode RPC URL with an embedded token. Anyone who receives this folder can import that key and drain the wallet. Act now: (1) move any funds out of that wallet and treat it as fully compromised, (2) delete the key from all seven files and read it from the `WALLET_PRIVATE_KEY` environment variable instead, (3) rotate the Helius, Birdeye, and QuikNode keys. The `.env.example` files themselves are clean placeholders — the leak is only in these non-env files.

🚨 Verified real leak: the ASES `.env.example` contains a real-looking GitHub token (`ghp_…`), reused in both `GITHUB_TOKEN` and `OPENAI_API_KEY`. Revoke that token now (GitHub → Settings → Developer settings → Personal access tokens) and replace both lines with placeholders. A live token in a shared zip is exposed even before any sale.

Before you zip anything for a buyer:

- **ASES** — remove the real GitHub token from `.env.example` (both `GITHUB_TOKEN` and `OPENAI_API_KEY` lines) and revoke it.
- **AI Video Monetizer** — the CLEAN zip already has no `client_secret.json`; keep it that way (buyers download their own from Google Cloud).
- **SafeScreen** — verified clean; the Kaggle credentials in `nsfw_fix.py` are already read from environment variables (empty defaults). Nothing to strip.
- **Solana Sniper** — delete the hardcoded wallet private key and API keys from `check_balance.cjs`, `monitor_progress.cjs`, `start_bot.ps1`, `start_bot_daemon.ps1`, `temp.txt`, and `README.md` (7 spots); read `WALLET_PRIVATE_KEY` from env; rotate the Helius/Birdeye/QuikNode keys. (No live `.env` / `.pem` exist — the `.gitignore` is correct, but these files aren't covered by it.)
- **Rotate/revoke** every key that was ever committed, even after deleting the files.

---

1. GODSHAND — Cinematic AI Film Kit

What it is: turns character reference images + a story into graded, motion-consistent film shots, then stitches them into a finished clip with an original score. The kit ships with **three ways to run it**.

Workflow A — `run.py` (Lightning.ai / paid GPU, cost-gated)

The kit's own documented flow (`RUN.md`). Safe by default: it audits, renders one sample, and projects cost before spending.

```
# SAFE by default: audit + 1 sample + cost projection, then S
TOP
python3 run.py
# When happy with the projection, render the full clip:
CONFIRM_FULL_RUN=1 MAX_BUDGET_USD=8 AUTO_SHUTDOWN=1 python3 r
un.py
```

- Pick the GPU profile in `config_confrontation.yaml` → `active_profile` (default is a Lightning A100/H100 profile).
- On free GPU hours, set that profile's `rate_per_gpu_hr: 0.0` so the cost meter reports ₹0.
- Render is resumable — finished shots auto-skip.
- Output: `output_confrontation/THE_CONFRONTATION_CLIP.mp4` (1280×720 ×2 → **1440p**).

Workflow B — `kaggle_run_confrontation.ipynb` (free Kaggle T4)

1. Upload the kit ZIP as a Kaggle Dataset (must contain a folder with `src/`).
2. New Notebook → File → Import Notebook → upload `kaggle_run_confrontation.ipynb`.
3. Add Input → attach the kit dataset (+ character refs).
4. Accelerator = **GPU T4 ×2**, Internet = **On**.
5. Run All. It copies the kit into `/kaggle/working`, downloads the Wan2.1-S2V Q8 GGUF (~14 GB), launches headless ComfyUI on `127.0.0.1:8188`, and renders. Set `active_profile: kaggle_2xt4`.

Workflow C — `rainman-wan-multichar.ipynb` (custom 2-minute multi-character film)

Same Kaggle setup as B (kit dataset + character refs, GPU T4 ×2, Internet On, Run All).

- Renders shot-by-shot (~8–9 min/shot), resumes automatically on re-run.
- Output: per-shot clips at `confrontation_work/<shot_id>/final.mp4`; the notebook stitches the full film + score automatically.
- Expect ~4.5–5.5 hours for the ~2-minute (28-shot) cut.

⚙️ The truth about Kaggle render speed: even though you select "T4 ×2", the Kaggle flows render on one T4 at a time by design. Kaggle's two GPUs share ~32 GB host RAM, and a RAM guard allows only a single render server (two would OOM). Do NOT run two notebooks pinned to each GPU — it wastes your weekly quota.

Free HD upscale: use `upscale_hd_kaggle.ipynb` (Accelerator = None/CPU, Internet = On). It uses **EDSR** (falls back to Lanczos only if the model can't download — confirm the printed method is `edsr`).

Honest limits: stylized AI footage with occasional artifacts. Guided notebooks/scripts, not fully hands-free software.

---

2. Convergence Framework

What it is: a rigorous thinking framework + LLM prompts for defining objectives, reviewing code, and debugging complex systems.

How to use:

1. Read `Convergence_Under_Constraint.pdf` (the core method).
2. Work through `README_objective_function.md` (Steps 0–8) to formalize goals.
3. Use `SystemPrompt_Code_Convergence_Deterministic.md` as a system prompt.
4. Review code with `AI_Code_Review_Prompt_v2.md`.
5. Study the `Trading_Bot_Convergence_Framework_v3.md` worked example.

ℹ️ Packaging tip: ship the clean `ConvergenceFramework_Kit/` folder to buyers — not `ConvergenceFramework_ALL/`, which also contains your LinkedIn drafts and backup CSVs.

Honest limits: a framework for *thinking* — not an auto-code generator. Requires human judgment.

---

3. AI Video Monetizer

What it is: an end-to-end pipeline for faceless content — pulls ideas from a Google Sheet, generates AI video (Runway/Luma/Kling), and handles posting and sales/DM automation.

Setup:

1. In Google Cloud Console → Credentials → create an OAuth 2.0 Client ID (Desktop app) → download the JSON as `client_secret.json`. Enable the Sheets and Drive APIs.
2. Authenticate: `python -m google_workspace.scripts.setup --client-secret ./client_secret.json`, then follow the auth-URL flow.
3. Copy `.env.example` → `.env` and fill in your keys.
4. Edit prompts in `config/video_prompts.json`.

Honest limits: needs the buyer's own API keys and configuration — not plug-and-play. Runs entirely on the buyer's own accounts (CLEAN build ships no seller credentials).

---

4. ASES — Autonomous Software Engineering System (v3)

What it is: a multi-agent system that plans, codes, tests, reviews, and delivers apps. Pipeline: Planner → Coder → Executor → Reviewer (feedback loop, max 5 iterations), running real `npm test` / `pytest` in Docker sandboxes and delivering a GitHub repo.

Setup (verified against the shipped files):

1. `cp .env.example .env`, then add your own keys (revoke the leaked token first — Section 0).
2. Start dev stack: `docker compose -f docker-compose.dev.yml up -d` (prod: `docker compose -f docker/docker-compose.yml up -d`).
3. Health check: `curl http://localhost:8000/health`.
4. Submit a task: `POST http://localhost:8000/dev-task` with headers `x-tenant-id` + `x-api-key`; poll `GET /jobs/{execution_id}`.

🏗️ Honesty label (keep this): ASES is complete but not production-hardened. Sell it as a working boilerplate/starter clearly marked "not production-hardened," not a finished SaaS.

---

5. SafeScreen — Content Moderation

What it is: a trained NSFW/toxic detector (`toxic_detector.onnx` + OCR) with mobile export (ONNX / TFLite / PyTorch-Mobile), ~30–50 ms on-device.

Quick start: run the collectors, train on safe/unsafe samples, then export the mobile model.

⚠️ Honest limit: a detector model + pipeline, not a full moderation product. ✅ Verified: all 64 scripts were scanned — no hardcoded secrets, and the Kaggle credentials in `nsfw_fix.py` are already read from environment variables. Nothing to strip.

---

6. Solana Sniper — Paper-Trading Study + Source

What it is: a 6-month paper-trading study with full source (React/TS dashboard, risk manager, Monte-Carlo analysis in `REAL_WORLD_RISK_ANALYSIS.md`).

Quick start: `npm run build`, then start the services (ML scorer, scanner, dashboard) in **paper mode**.

🚫 Critical positioning (do NOT change): sell this ONLY as a study + source-code learning artifact. Never pitch it as a live money-maker or "profitable trading bot." Ship it in paper mode. Honest framing is your legal protection. ⚠️ Verified: this project ships with a live wallet private key and live API keys hardcoded in 7 files — you MUST complete Section 0 (delete the key, read from env, rotate keys, move funds) before it ever leaves your machine.

Packaging as a single product (for resale)

```
Master_Collection_2999INR/
├── GODSHAND/            (kit + run.py + notebooks)
├── Convergence/         (ship the _Kit folder only)
├── AI_Video_Monetizer/  (CLEAN build, no client_secret.json)
├── ASES/                (token revoked + placeholder; "not hardened")
├── SafeScreen/          (verified clean)
├── Solana_Sniper/       (paper mode; wallet key removed)
├── Master_Setup_Guide.pdf   ← this document
├── LICENSE.txt              (full white-label rights granted)
└── README.md                (overview + pricing)
```

Seller checklist:

1. URGENT — move funds out of the exposed Solana wallet, delete the hardcoded private key from all 7 files, and rotate the Helius/Birdeye/QuikNode keys.
2. Revoke the leaked ASES GitHub token and replace with a placeholder.
3. Strip + rotate any other secrets across all projects.
4. Clean-room test each project yourself.
5. Zip using the structure above.
6. Build a sales page with screenshots/demos.
7. Deliver via Google Drive/Dropbox after payment confirmation.
8. Price at ₹2,999 (one-time, lifetime access + rights).

> 💡 Strategy note (your call, already made): a ₹2,999 all-in white-label bundle is cheaper than the projects sold individually (~₹5,500 combined) and lets buyers resell your work. A deliberate volume/survival-pricing choice.